

Reporte de la práctica

Temporización no bloqueante: tres LEDs con delay() contra tres LEDs con millis()

1. Datos generales

Materia	Sistemas Programables
Institución	Instituto Tecnológico de Mazatlán (TecNM)
Alumno	Jesús Antonio Rodríguez Peraza — L231000105
Práctica	Práctica 2.1.1 — Temporización no bloqueante. Parte 1: delay() (antipatrón). Parte 2: millis() (solución) con reto opcional del serial
Placa utilizada	Arduino UNO R4 WiFi
Componentes	Arduino UNO R4 WiFi, 3 LEDs, 3 resistencias de 220 Ω , protoboard, cables Dupont
Programas	ParpadeoDelay.ino (Codigo/ParpadeoDelay/) y ParpadeoMillis.ino (Codigo/ParpadeoMillis/), ambos para el mismo circuito
Repositorio	github.com/Jesus-Jazisenot/Universidad → Sistemas-Programables/Practica-2-Temporizacion-No-Bloqueante-delay-vs-millis

2. Objetivo

Hacer que tres LEDs parpadeen con periodos distintos (500, 1000 y 1500 ms) de dos formas sobre el mismo circuito: primero con delay(), para comprobar con los propios ojos que los retardos bloqueantes detienen por completo la ejecución y no permiten que varias tareas convivan en un solo loop(); y después con millis(), sustituyendo las esperas por comparaciones, organizando las tareas en un struct y un arreglo, y comprobando con una cuarta tarea (mensaje serial cada 3 s) que las demás no se ven afectadas. Al final, comparar las dos formas de trabajar.

3. Diagrama del circuito (conexiones)

LED	Pin Arduino	Resistencia	Periodo pedido
LED 1	8	220 Ω a GND	500 ms
LED 2	9	220 Ω a GND	1000 ms
LED 3	10	220 Ω a GND	1500 ms

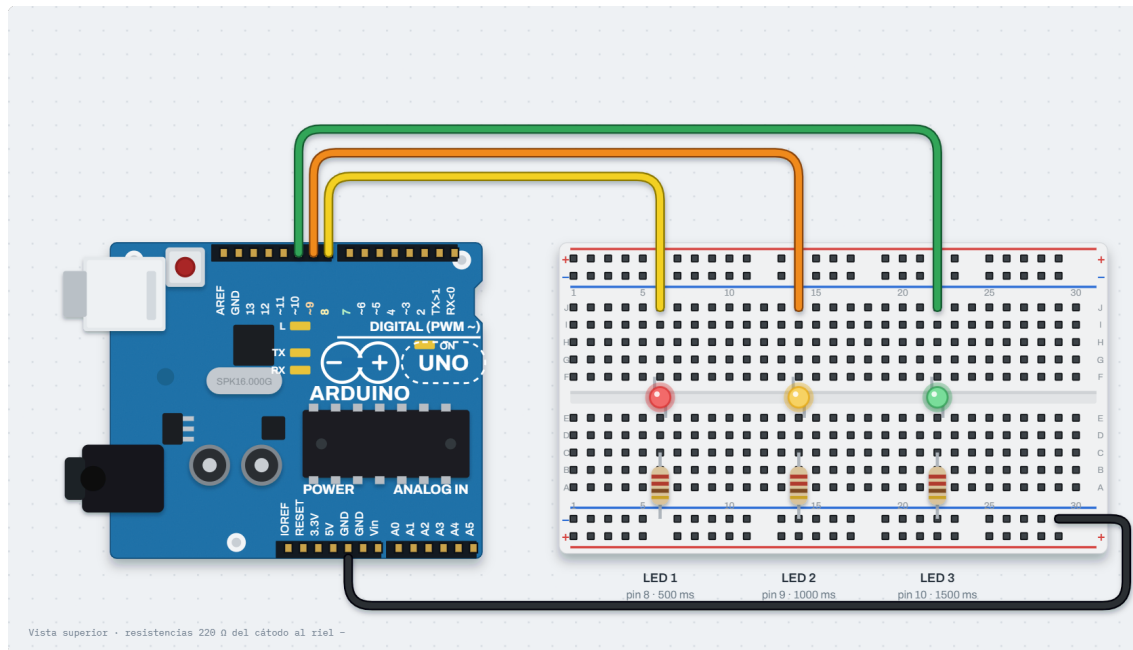


Figura 1. Diagrama de conexiones; el mismo circuito para las dos partes.

4. Procedimiento

Parte 1 — delay()

1. Se armó el circuito en la protoboard: cada LED del pin correspondiente (8, 9 y 10) al ánodo, y el cátodo por una resistencia de $220\ \Omega$ a GND.
2. Se cargó ParpadeoDelay.ino. El programa enciende y apaga cada LED con digitalWrite() y delay(), uno tras otro, con los tiempos pedidos.
3. Se observó el parpadeo, se grabó el video y se contó cuántas veces enciende cada LED en un ciclo completo para compararlo contra lo pedido.

Parte 2 — millis()

1. Sin tocar el circuito, se cargó ParpadeoMillis.ino. Cada LED es una entrada del arreglo leds[] con su pin, su periodo, el instante de su último cambio y su estado.
2. En cada vuelta del loop() se lee millis() una sola vez y, por cada tarea, se evalúa if (ahora - ultimoCambio >= periodoMs); si se cumple, se invierte el LED y se anota el instante.
3. Se agregó la cuarta tarea del reto: un mensaje en el Monitor Serie (9600 baudios) cada 3000 ms con el número de mensaje, el tiempo y el estado de los tres LEDs.
4. Se observó el parpadeo simultáneo, se leyó el Monitor Serie y se grabó el video.
5. Se compararon los dos comportamientos con el mismo intervalo de 6 s (sección 6).

5. Tablas de tiempos de cada parte

Parte 1 — delay(): una sola secuencia de 6000 ms en la que cada LED enciende una vez, uno después del otro:

Tiempo (ms)	LED1	LED2	LED3	Instrucción en ejecución
0 – 500	ON	off	off	delay(500)
500 – 1000	off	off	off	delay(500)
1000 – 2000	off	ON	off	delay(1000)
2000 – 3000	off	off	off	delay(1000)
3000 – 4500	off	off	ON	delay(1500)
4500 – 6000	off	off	off	delay(1500)

Parte 2 — millis(): estados de los tres LEDs durante los primeros 3 s. El patrón se repite cada 3000 ms (mínimo común múltiplo de los periodos) y ahí cae el mensaje serial:

Tiempo (ms)	LED1 (500)	LED2 (1000)	LED3 (1500)	LEDs encendidos
0	off	off	off	0
500	ON	off	off	1
1000	off	ON	off	1
1500	ON	ON	ON	3
2000	off	off	ON	1
2500	ON	off	ON	2
3000	off	ON	off	1

** Tiempos verificados con los videos y, en la Parte 2, con los instantes que imprime el Monitor Serie. La Parte 1 no usa el puerto serie.*

6. Comparación de las dos formas de trabajar

Aspecto	Parte 1 — delay()	Parte 2 — millis()
Ritmo de cada LED	Ninguno cumple su periodo: todos encadenados en un ciclo de 6 s	Cada uno cumple el suyo: 500 / 1000 / 1500 ms
Encendidos en 6 s (LED1 / LED2 / LED3)	1 / 1 / 1	6 / 3 / 2 (lo pedido)
LEDs encendidos a la vez	Nunca más de uno	Hasta los tres (a los 1500 ms)
Duración de una vuelta del loop()	6000 ms	Microsegundos
¿Atiende un botón o el serial mientras "espera"?	No: está bloqueado dentro del delay()	Sí: nunca espera, solo compara
Agregar una tarea nueva (reto: serial cada 3 s)	Hay que reescribir la secuencia y aun así llega tarde	Un renglón más, sin tocar las otras
Uso del procesador	Ocupado "sin hacer nada"	Libre entre comparaciones
Desbordamiento de millis()	No aplica	Resuelto por la resta en unsigned long
Complejidad del código	Más corto, pero incorrecto	Un struct y un for; correcto y escalable

Aspecto	Parte 1 — delay()	Parte 2 — millis()
Dónde sirve	Solo si el programa hace una sola cosa	Cualquier sistema que atienda varias cosas a la vez

7. Salida del Monitor Serie de la Parte 2 (9600 baudios)

```

Parpadeo con millis(): 500 / 1000 / 1500 ms
Mensaje #1  t = 3000 ms  LEDs: 010
Mensaje #2  t = 6000 ms  LEDs: 000
Mensaje #3  t = 9000 ms  LEDs: 010
Mensaje #4  t = 12000 ms LEDs: 000
Mensaje #5  t = 15000 ms LEDs: 010
Mensaje #6  t = 18000 ms LEDs: 000

```

La columna LEDs (por ejemplo 010) es el estado de LED1, LED2 y LED3 en el instante del mensaje: a los 3000 ms LED1 lleva 6 cambios (apagado), LED2 lleva 3 (encendido) y LED3 lleva 2 (apagado). Transcripción de lo que imprime el programa.

8. Observaciones sobre el comportamiento del sistema

- Parte 1: los LEDs encienden por turnos (rojo, amarillo, verde) y el ciclo completo tarda 6 s, exactamente la suma de los seis delay() del loop(). En ningún momento hay dos encendidos a la vez.
- Parte 1: el programa compila y "funciona" sin ningún error; el problema solo se ve al comparar el ritmo real contra el pedido. Es un error de diseño, no de sintaxis.
- Parte 2: los tres LEDs parpadean al mismo tiempo, cada uno con su periodo; se observan momentos con dos LEDs encendidos y a los 1500 ms los tres a la vez. El patrón se repite cada 3 s.
- Parte 2: el mensaje serial cae exactamente en los múltiplos de 3000 ms sin desfase acumulado, y agregarlo no alteró el ritmo de ningún LED.
- Parte 2: cada LED es una entrada de un struct Parpadeo en un arreglo; agregar un LED es agregar un renglón, no reescribir el loop().
- Mismo circuito en las dos partes: la diferencia de comportamiento se debe únicamente al programa.

9. Conclusiones

delay() no significa "espera 500 ms para este LED": significa "detén todo el programa 500 ms". Mientras el procesador está dentro de un delay() no revisa ningún otro LED, ni un botón, ni el puerto serie; por eso en la Parte 1 los tres LEDs quedaron encadenados en una sola secuencia de 6 s y ninguno cumplió su periodo. En la protoboard se vio claro: encienden por turnos, nunca dos a la vez.

Con millis() la diferencia se ve a simple vista: hay instantes con dos y hasta tres LEDs encendidos al mismo tiempo, y cada uno lleva su ritmo. Con el mismo circuito, lo único que cambió fue la forma de esperar. El cambio es de mentalidad más que de código: en vez de "espera X ms" el programa dice "revisa si ya toca". Cada tarea recuerda cuándo actuó (ultimoCambio) y compara ahora - ultimoCambio >= periodo; como nadie detiene al procesador, el loop() gira miles de veces por segundo y todas las tareas se atienden.

La cuarta tarea del reto (mensaje serial cada 3 s) entró sin tocar una sola línea de los LEDs: solo se agregó otra comparación con su propio periodo. Eso demuestra que la estructura es realmente no bloqueante y que agregar tareas escala. Con delay() esa misma tarea habría obligado a reescribir

todo el loop()) y aun así habría llegado tarde.

La resta en unsigned long resuelve el desbordamiento de millis() a los ~49.7 días: cuando el contador da la vuelta, la diferencia también da la vuelta y sigue siendo la diferencia real. Escribir ahora `>= ultimoCambio + periodo` sí fallaría en ese momento.

El patrón tiene un límite: sigue siendo multitarea cooperativa. Si una tarea tarda mucho (un cálculo largo o un `delay()` escondido en una librería) retrasa a todas las demás. Para garantías de tiempo estrictas se necesitan interrupciones o temporizadores por hardware. Aun así, para un sistema embebido real (sensores, comunicación, botones) la regla queda clara: no esperar nunca; el `delay()` es un antipatrón.

10. Evidencia del armado

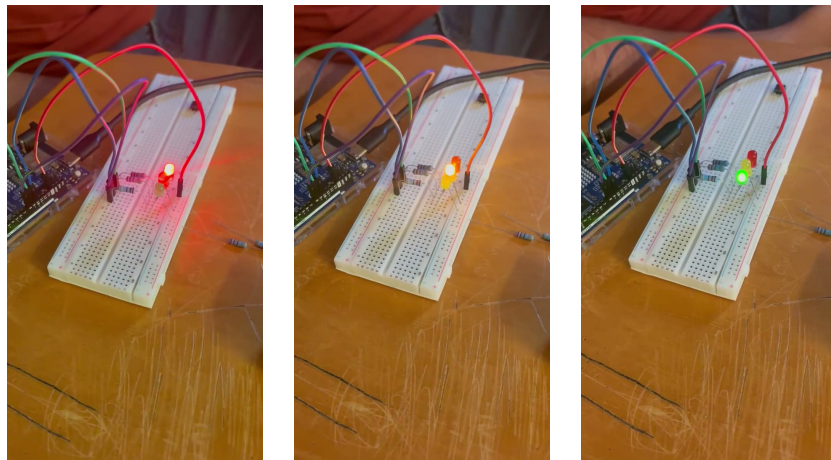


Figura 2. Parte 1 (delay): los LEDs encienden uno después del otro, nunca dos a la vez.

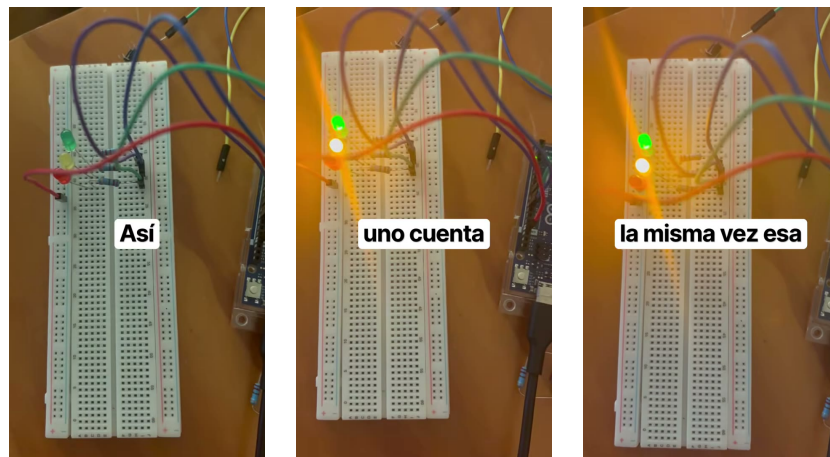


Figura 3. Parte 2 (millis): dos LEDs encendidos al mismo tiempo, cada uno con su ritmo.

Video Parte 1 (delay): <https://www.youtube.com/shorts/sUPkjt看an6g>

Video Parte 2 (millis): https://www.youtube.com/shorts/pJ0OG_M3bCs